

Disciplina:

Machine Learning

Professor: Maurício Rodrigues da Silva



Descrição da Disciplina

Processo de aprendizagem de máquina. Feature engineering. Técnicas e algoritmos de aprendizado supervisionado e não-supervisionado. Modelos ensemble. Métricas e avaliação de modelos. Explicabilidade, interpretação de métricas e validação cruzada com IA Generativa. Prompts para identificação e criação de features relevantes. Projeto prático.



As 5 Unidades

- | | | |
|---|---|----|
| 1 | Ciclo de Vida de ML e Feature Engineering | 4h |
| 2 | Aprendizado Supervisionado: Regressão e Ensembles | 4h |
| 3 | Classificação, Métricas e Explicabilidade | 8h |
| 4 | Aprendizado Não-Supervisionado: Agrupamento | 4h |
| 5 | Regras de Associação e Projeto Prático | 4h |

Introdução: Avaliação da Disciplina

Os 100 pontos serão distribuídos da seguinte forma:

60

PONTOS

Atividades Práticas de Fixação

40

PONTOS

**Projeto Final
Aplicação de GenAI end-to-end**

Nosso Estudo de Caso: Churn de Telecom

O problema

- Uma operadora quer prever quais clientes têm risco de cancelar (churn).
- Meta de negócio: Recall $\geq 0,75$, ou seja, encontrar 75% dos futuros cancelamentos.

Por que este dataset

- Mistura variáveis numéricas e categóricas, como qualquer dado real.
- Serve para regressão, classificação, clustering e associação: as 5 unidades.

Seu projeto

- No Trabalho Orientado, você aplica o mesmo ciclo a um domínio à sua escolha.
- Finanças, e-commerce, saúde, meio ambiente: a técnica é a mesma.

UNIDADE 1 • 4 h/a



Ciclo de Vida de ML e Feature Engineering

Do problema de negócio à produção, com IA Generativa como copiloto.

1.1 Ciclo de Vida de ML

1.2 Ferramentas e EDA

1.3 Feature Engineering com GenAI

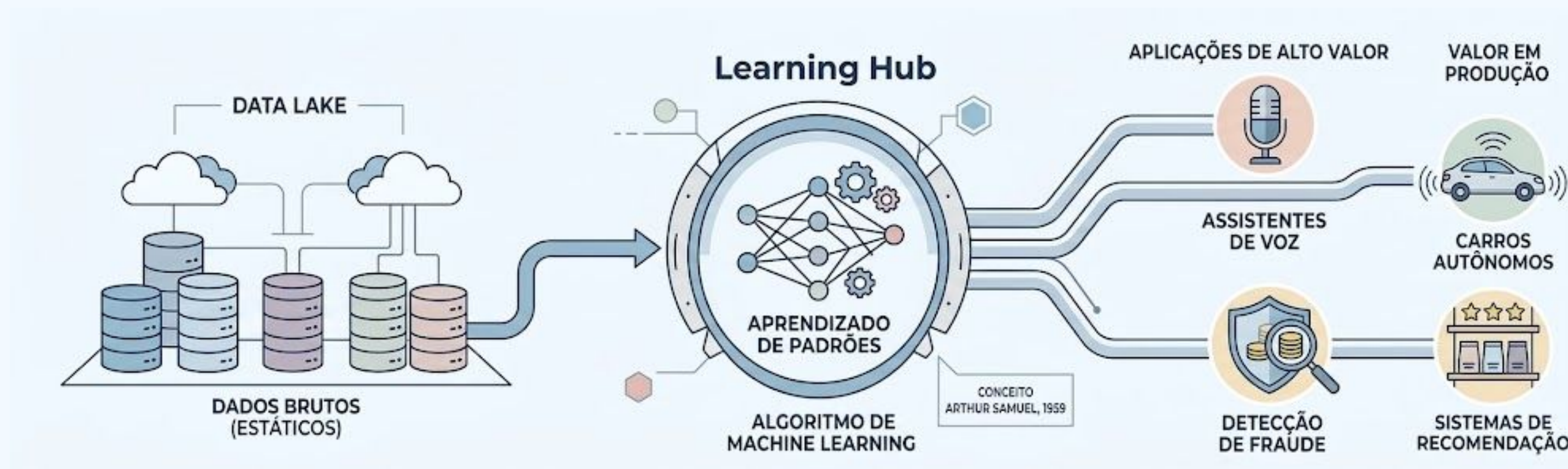
1.4 Pré-processamento

O Que Vamos Construir Hoje

- 1.1 Ciclo de Vida de ML: do problema de negócio à produção
- 1.2 Ferramentas e EDA: Python, Jupyter, distribuições, outliers
- 1.3 Feature Engineering com GenAI: prompts para brainstorming de features
- 1.4 Pré-processamento: dados faltantes, encoding, data leakage
-  Prática 01: EDA + Feature Engineering no dataset de Churn

Por Que Machine Learning Agora?

- **Não vivemos mais a era de "ter dados"** : vivemos a era de extrair valor em produção. Dados parados em um data lake não geram receita.
- **Machine Learning** = sistemas que aprendem padrões a partir de dados, sem serem explicitamente programados para cada regra (Arthur Samuel, 1959).
- **Hoje** , ML é a base de assistentes de voz, carros autônomos, detecção de fraudes e sistemas de recomendação.

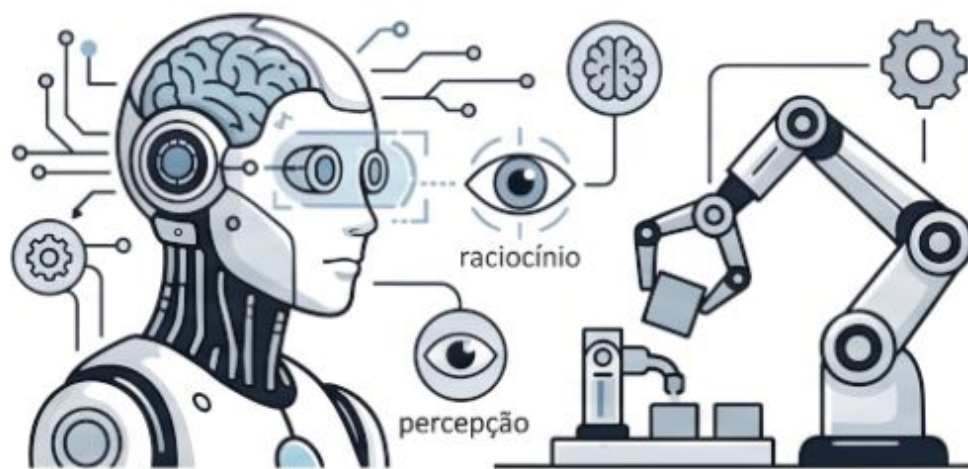


💡 O problema deixou de ser "como processar dados", e passou a ser "como extrair valor em produção".

IA, ML, Deep Learning e IA Generativa

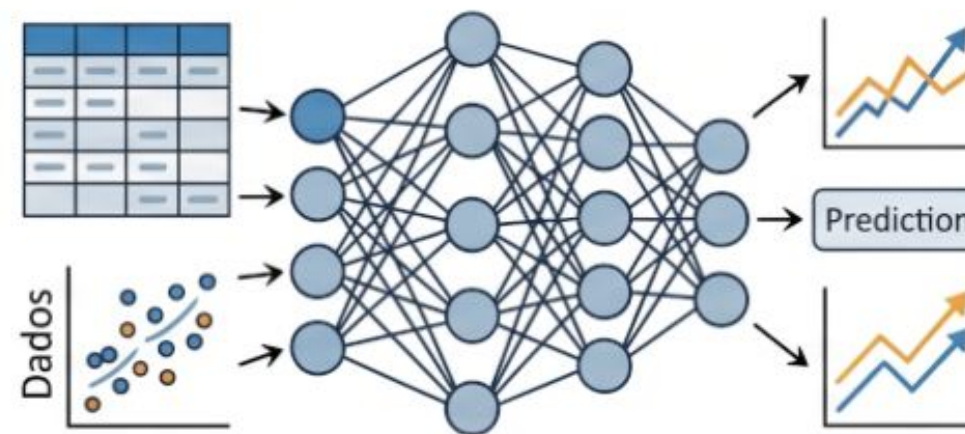
Inteligência Artificial

- Campo mais amplo: sistemas que raciocinam, aprendem, percebem e agem de forma autônoma.



Machine Learning

- Subcampo da IA que aprende padrões a partir de dados: nosso foco no curso.

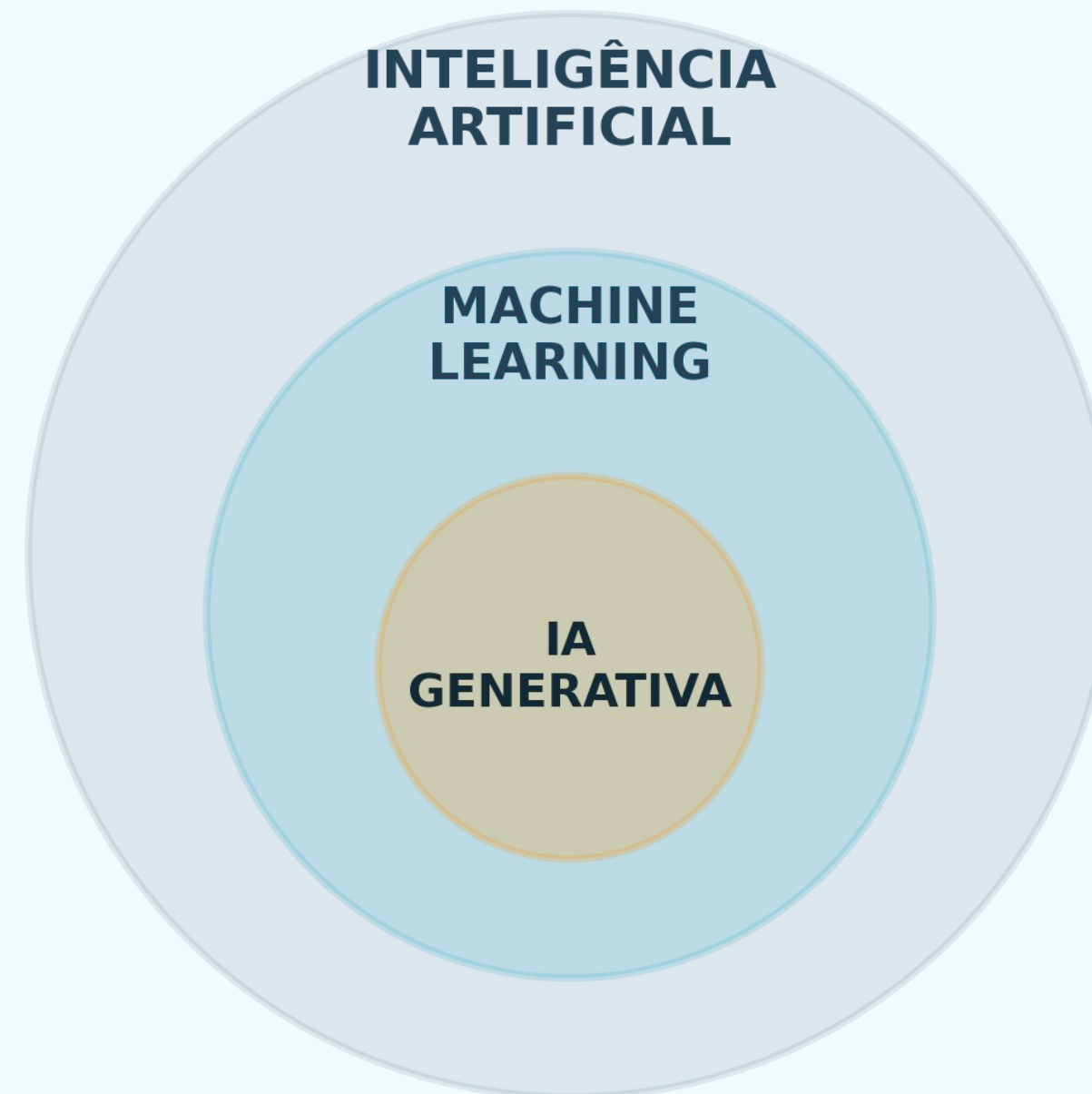


IA Generativa

- Camada que se aplica sobre o ML: gera hipóteses, cria features e interpreta resultados: nosso copiloto.



Da IA à IA Generativa, em Camadas



Cada camada é um subconjunto da anterior: toda IA Generativa é ML, mas nem todo ML é IA Generativa.

Por Que Projetos de ML Falham?

Qualidade de dados

- Dados sujos geram modelos ruins.
- Algoritmo avançado não salva entrada de má qualidade.

Lacuna de talentos

- Poucos profissionais qualificados.
- AutoML e GenAI ajudam a automatizar tarefas repetitivas.

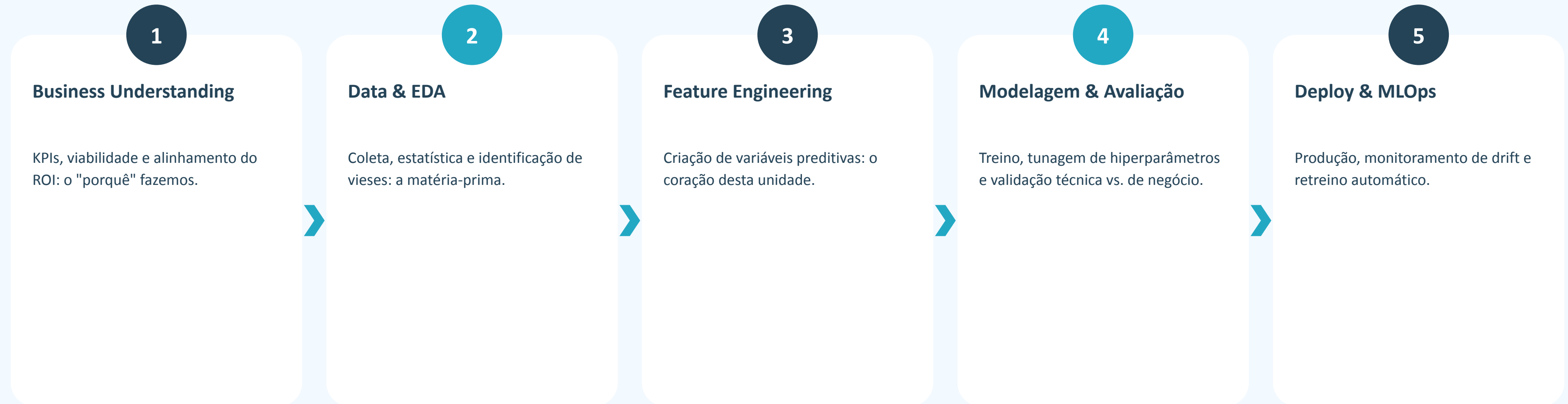
Transição de paradigma

- Migrar de uma cultura reativa (BI) para uma cultura preditiva.

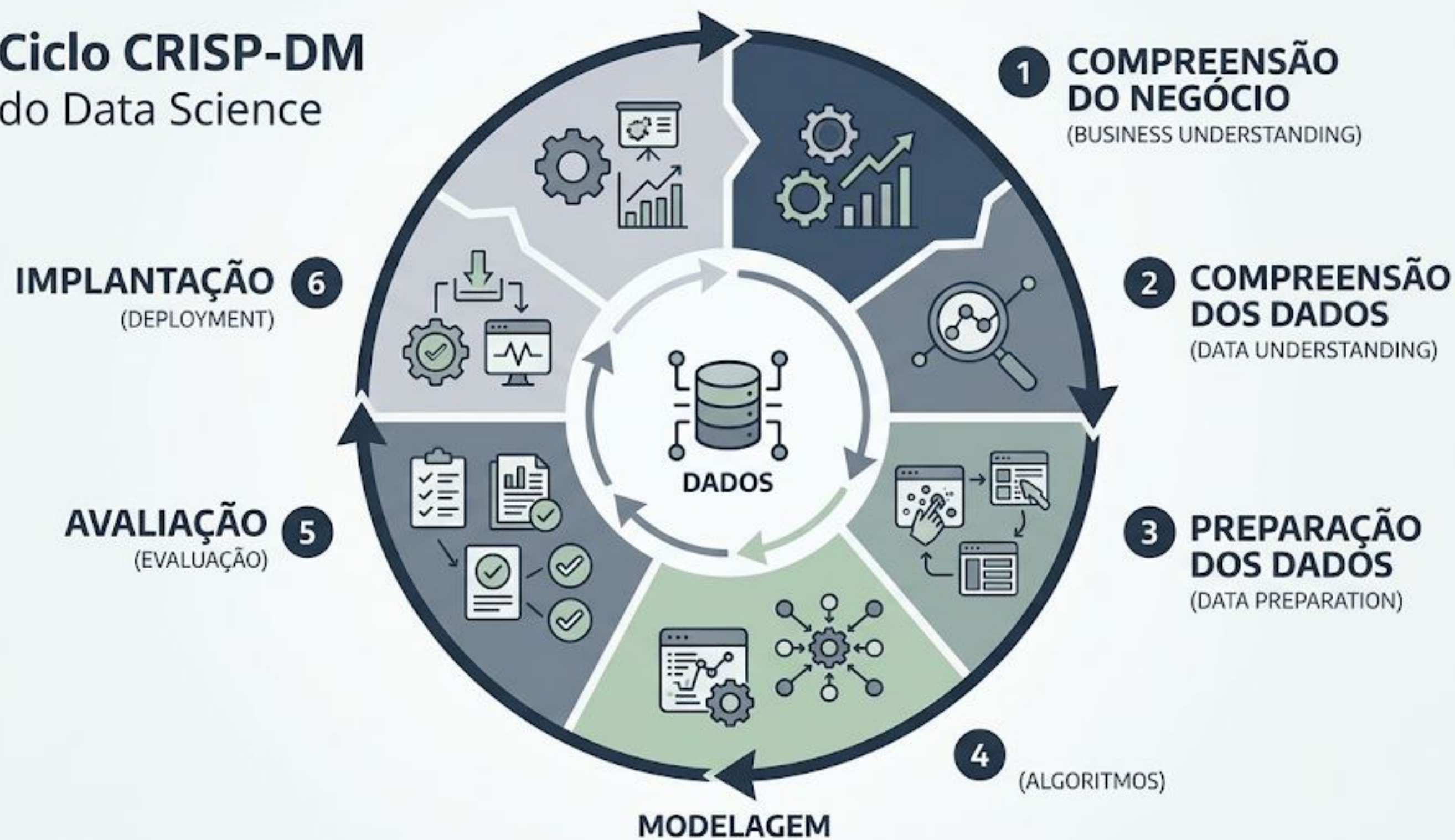
Governança e ética

- Viés, explicabilidade e privacidade.
- Modelo preciso mas injusto ou ilegal não serve.

O Ciclo de Vida de ML (MLOps)



Ciclo CRISP-DM do Data Science



Business Understanding: Churn de Telecom

Vamos usar este caso durante toda a unidade, e você aplicará o mesmo raciocínio no seu Trabalho Orientado.

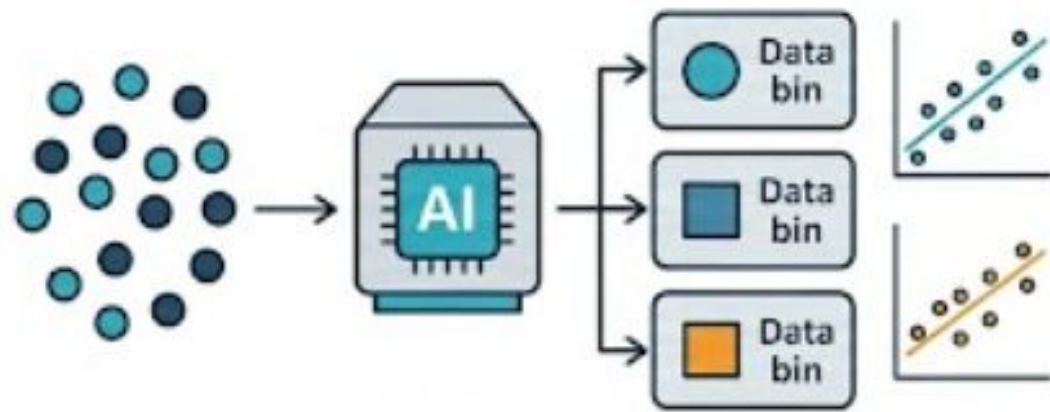
- **Problema:** como prever clientes com risco de cancelamento (churn)?
- **Objetivo:** intervir proativamente antes do cancelamento.
- **Critério de sucesso:** Recall $\geq 0,75$, ou seja, encontrar 75% dos clientes que vão cancelar.

► Toda feature que criarmos precisa responder: "isso ajuda a prever churn?" Se não ajudar, é ruído.

Que Tipo de Problema Estamos Resolvendo?

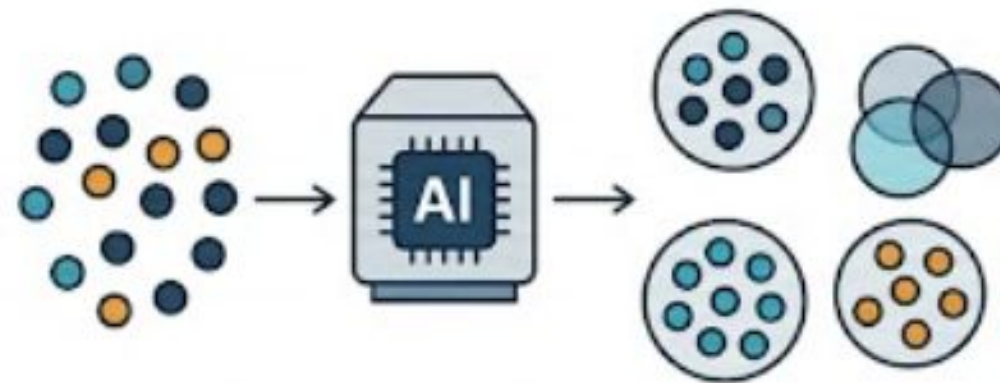
Supervisionado

- Tem rótulo (alvo) conhecido.
- Ex.: prever churn.
- Regressão e Classificação: Unidades 2 e 3.



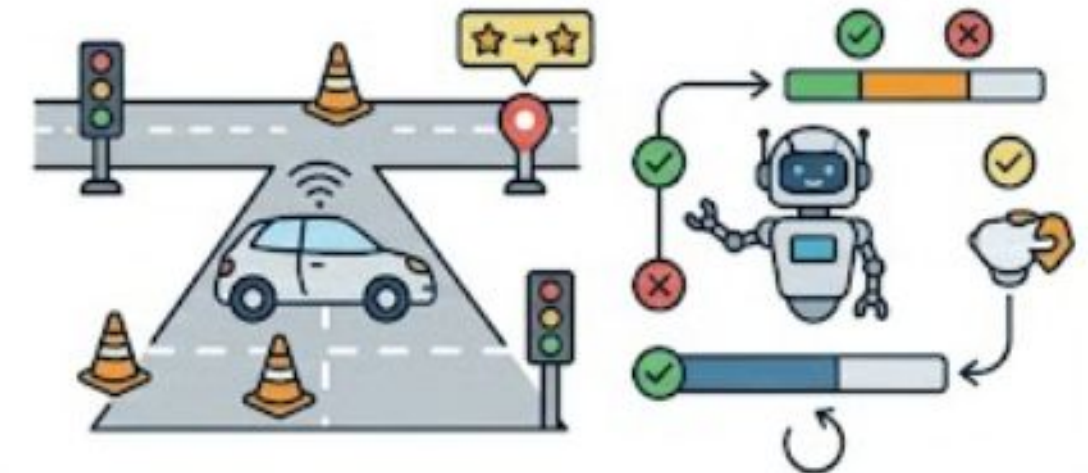
Não-Supervisionado

- Não tem rótulo.
- Ex.: segmentar clientes.
- Clustering e Associação: Unidades 4 e 5.



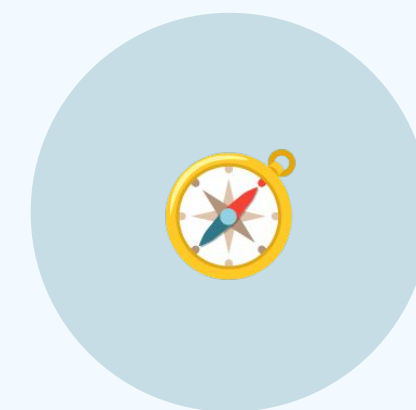
Por Reforço

- Um agente aprende interagindo com o ambiente.
- Ex.: carro autônomo.
- Fora do escopo do curso, mas vale conhecer.



A Unidade 1 Serve a Todos os Paradigmas

- Feature Engineering e pré-processamento são necessários em qualquer um dos três paradigmas.
- **Hoje, nosso foco prático é supervisionado:** o rótulo é a coluna Churn (Sim/Não).
- Mas as técnicas de EDA e criação de variáveis que veremos valem para todo o curso.



Dado Rotulado vs. Não Rotulado

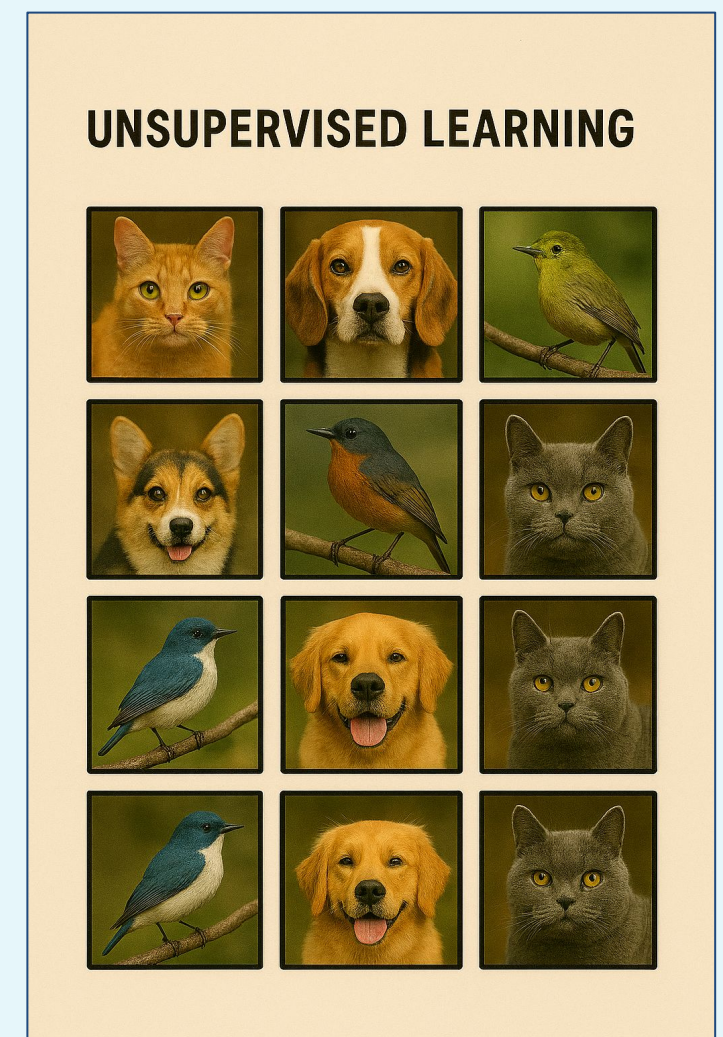
Dado Rotulado (Labeled Data)

- Já contém a "resposta correta", adicionada por um humano.
- Ex.: fotos marcadas como "gato", "cachorro" ou "pássaro".
- No nosso caso: a coluna Churn (Sim/Não).



Dado Não Rotulado (Unlabeled Data)

- Não possui tag, categoria ou valor-alvo associado.
- A maioria dos dados brutos da internet está nesse formato.
- O modelo precisa descobrir padrões por conta própria.



Rotulado vs. Não Rotulado, Lado a Lado

Rotulado — cada linha já tem a resposta certa

cliente	tenure	MonthlyCharge	Churn
7590-VHVEG	1	29.85	Sim
5575-GNVDE	34	56.95	Não
3668-QPYBK	2	53.85	Sim
7795-CFOCW	45	42.30	Não
9237-HQITU	8	70.70	Sim

Não rotulado — o modelo não sabe a resposta certa

cada ponto é um cliente, mas nenhum tem rótulo (Churn?) associado



À esquerda, cada cliente já tem a resposta (Churn); à direita, os mesmos clientes sem essa informação.

Onde o ML Já Está no seu Dia a Dia

Recomendação

- Streaming, e-commerce, playlists musicais.

Detecção de fraude

- Cartões, seguros, spend control corporativo.

Manutenção preditiva

- Sensores industriais prevendo falhas de equipamento.

Medicina personalizada

- Diagnóstico assistido e resposta a tratamentos.

Agricultura de precisão

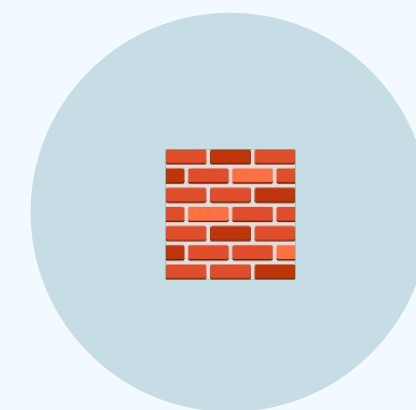
- Previsão de safra e uso otimizado de insumos.

Detecção de churn

- Nosso caso do dia: retenção proativa de clientes.

Tudo Depende de Feature Engineering de Qualidade

- Não importa se é uma rede neural profunda ou uma regressão simples.
- Se as features forem ruins, o modelo será ruim.
- Se as features forem boas, até um modelo simples pode ter performance excelente.



Ferramentas Python para ML

Manipulação

- Pandas
- NumPy



Visualização

- Matplotlib
- Seaborn



Modelagem

- Scikit-learn
- XGBoost



Copiloto GenAI

- ChatGPT / Claude
- GitHub Copilot



Datasets Públicos para Praticar

Fonte	Características	Diferencial
Kaggle Datasets (https://www.kaggle.com/datasets)	Maior plataforma de competições de ciência de dados	Comunidade ativa com notebooks e discussões prontas
Google Dataset Search (https://datasetsearch.research.google.com/)	Motor de busca dedicado a datasets na web	Ótimo para tópicos de nicho
UCI ML Repository (https://archive.ics.uci.edu/)	Repositório clássico, usado em livros e artigos	Datasets limpos, ideais para aprendizado
Dados.gov.br (https://dados.gov.br/)	Plataforma oficial do governo brasileiro	Fonte primária e confiável de dados públicos do Brasil

Data Acquisition: Telecom Churn (IBM/Kaggle)

customerID	tenure	MonthlyCharges	Contract	Churn
7590-VHVEG	1	29.85	Mensal	Sim
5575-GNVDE	34	56.95	Anual	Não
3668-QPYBK	2	53.85	Mensal	Sim

Repare: já há uma mistura de variáveis numéricas (tenure, charges) e categóricas (Contract, Churn): dado real é assim.

Coletando os Dados em Python

```
import pandas as pd

# Opção 1: arquivo baixado do Kaggle
df = pd.read_csv('telecom_churn.csv')

# Opção 2: direto de uma URL pública
# df = pd.read_csv('https://.../telecom_churn.csv')

df.shape
```

Este é o primeiro código que você vai rodar no notebook da Prática 01, antes de qualquer análise.

O Que é Análise Exploratória de Dados?

01 • Qualidade

- Valores ausentes e inconsistências.

02 • Distribuição

- Forma dos dados: normal ou assimétrica.

03 • Outliers

- Valores extremos: erro ou sinal?

04 • Correlação

- Relações entre variáveis.

df.describe(): Sua Primeira Visão Estatística

Estatística	O que significa
count	Quantos valores não nulos existem nesta coluna (ajuda a achar dados ausentes)
mean	Média: soma de todos os valores dividida pela contagem
std	Desvio padrão: o quanto os valores costumam se afastar da média
min	O menor valor observado na coluna
25% (Q1)	Primeiro quartil: 25% dos dados estão abaixo deste valor
50% (mediana)	Valor central: metade dos dados abaixo, metade acima
75% (Q3)	Terceiro quartil: 75% dos dados estão abaixo deste valor
max	O maior valor observado na coluna

Compare mean com 50% (mediana) logo de cara: quanto mais distantes, mais assimétrica é a distribuição.



Rodando o describe() em Python

```
df.describe()

# por padrão, describe() olha só colunas numéricas.
# para incluir as categóricas também (count, unique, top, freq):
df.describe(include='all')

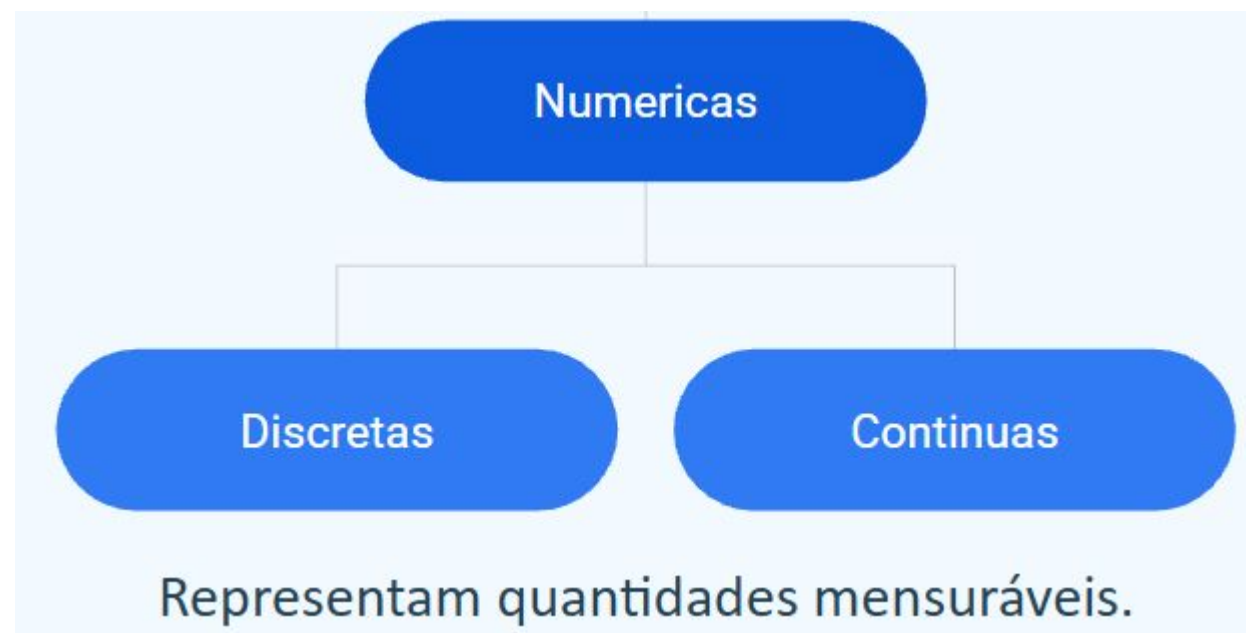
# describe() de uma única coluna:
df['MonthlyCharges'].describe()
```

Este é sempre o segundo comando que rodamos, logo depois de carregar o dataset.

Numéricas vs. Categóricas

Numéricas

- Discretas: contagens, valores inteiros (nº de filhos, acessos/dia).
- Contínuas: qualquer valor num intervalo (altura, salário, tenure).
- Média e desvio padrão fazem sentido.

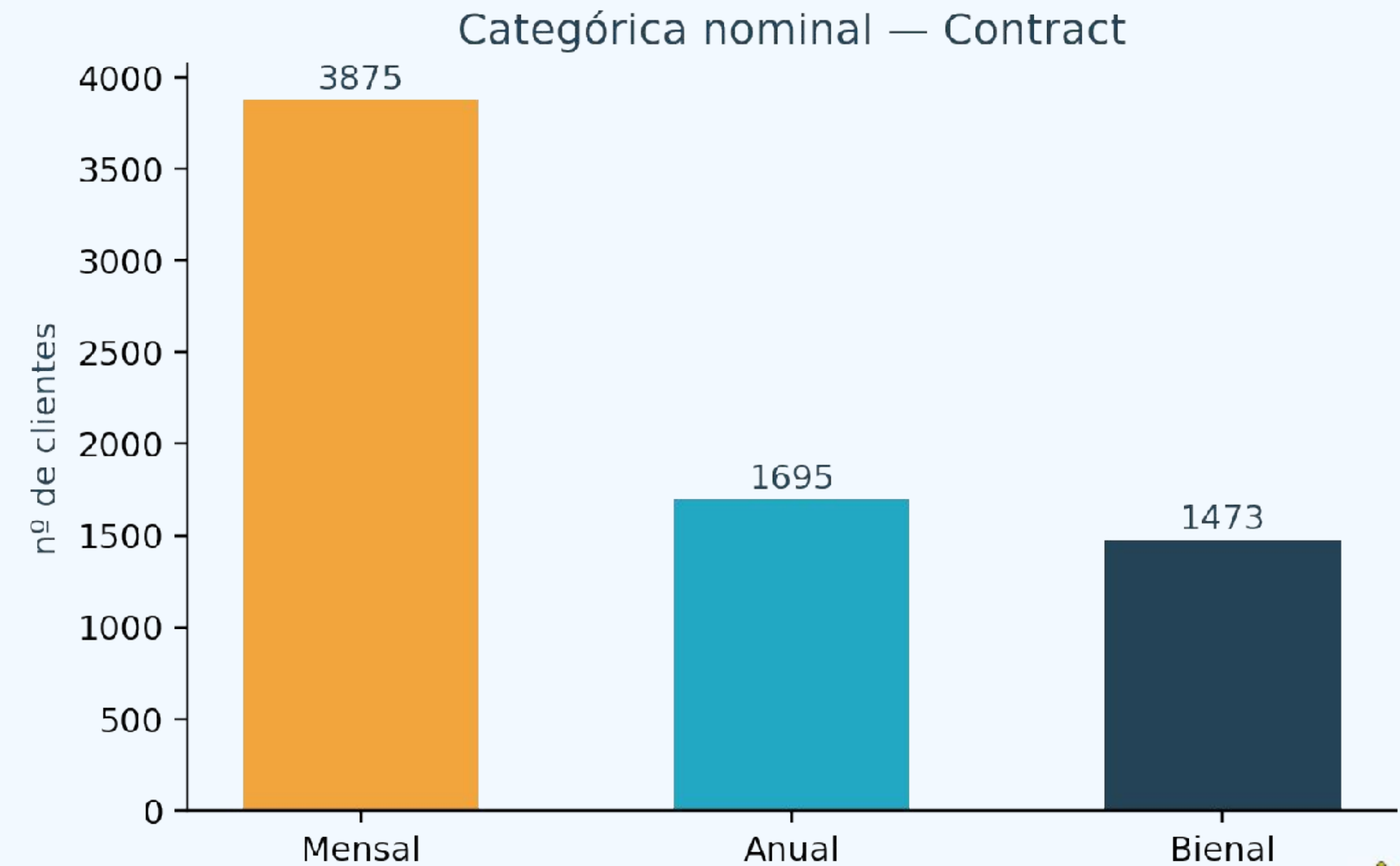
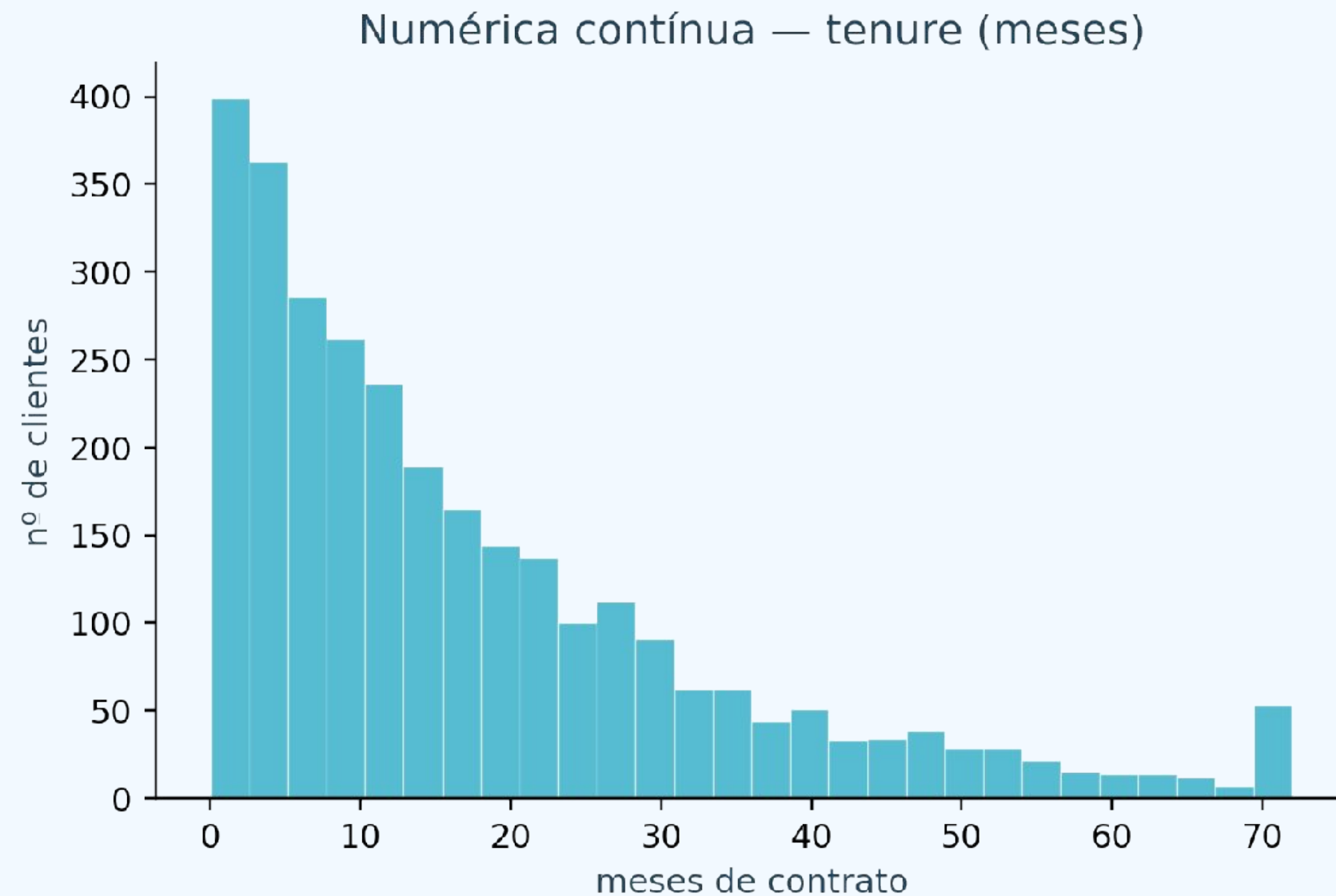


Categóricas

- Nominais: sem ordem natural (cor dos olhos, marca de celular).
- Ordinais: com ordem/ranking (escolaridade, satisfação).
- O tipo define encoding: One-Hot para nominais, ordinal para ordinais.



O Mesmo Dataset, Dois Tipos de Variável



tenure é numérica contínua (histograma); Contract é categórica nominal (contagem por categoria).

O Tipo da Variável Define a Técnica

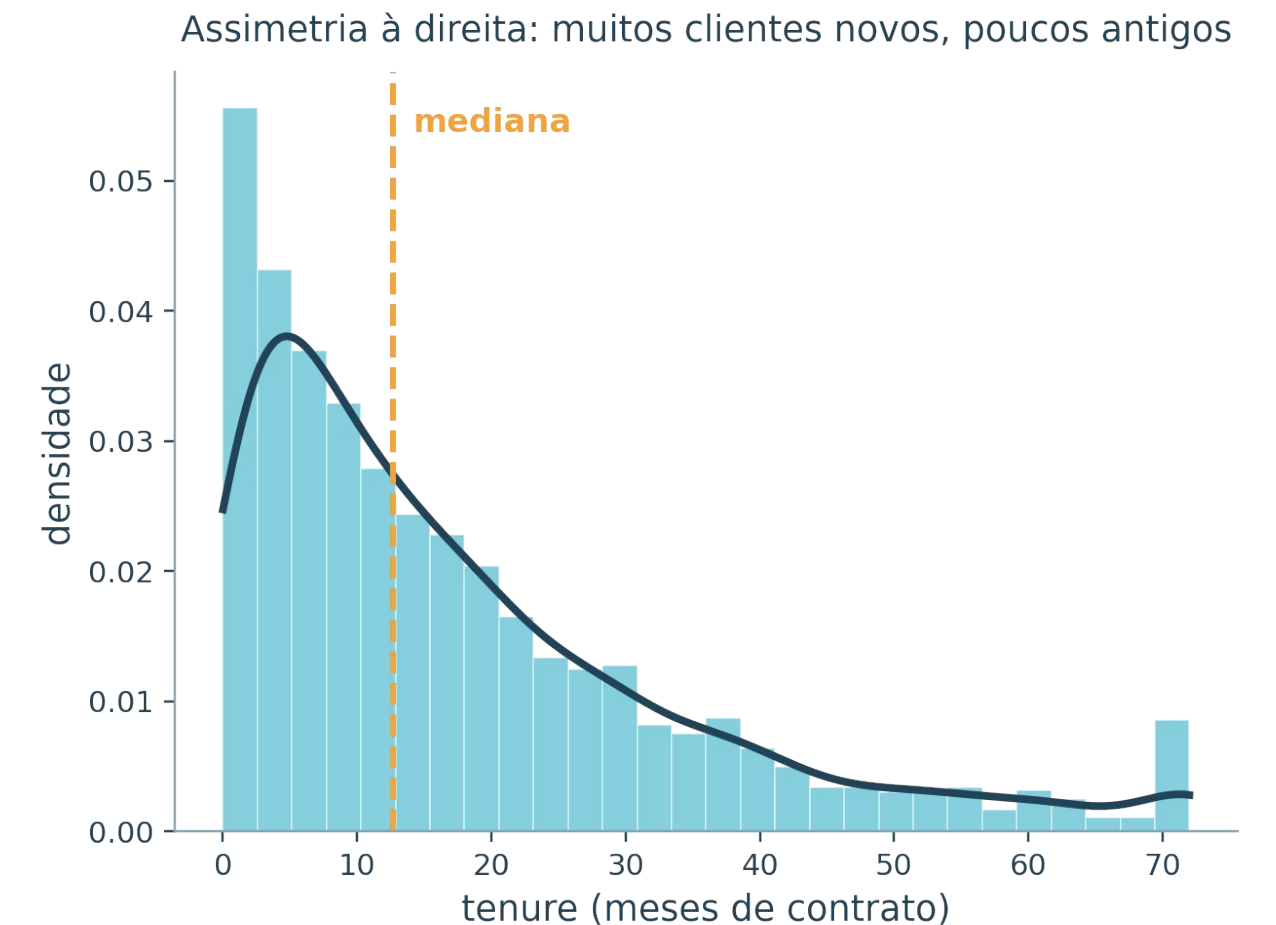
- Média faz sentido para variável numérica: não para categórica.
- O tipo define qual gráfico usar e como pré-processar.
- No nosso dataset: tenure é numérica contínua; Contract é categórica (com ordem implícita: Mensal < Anual < Bienal).



Análise de Distribuições

O histograma com curva de densidade (KDE) é nossa principal ferramenta.

- Os dados são simétricos ou assimétricos?
- Há múltiplos picos (multimodal)?
- **A relação entre média e mediana** já denuncia a assimetria: vamos ver isso no próximo slide.
- Precisamos de transformação (log, Box-Cox) antes de modelar?

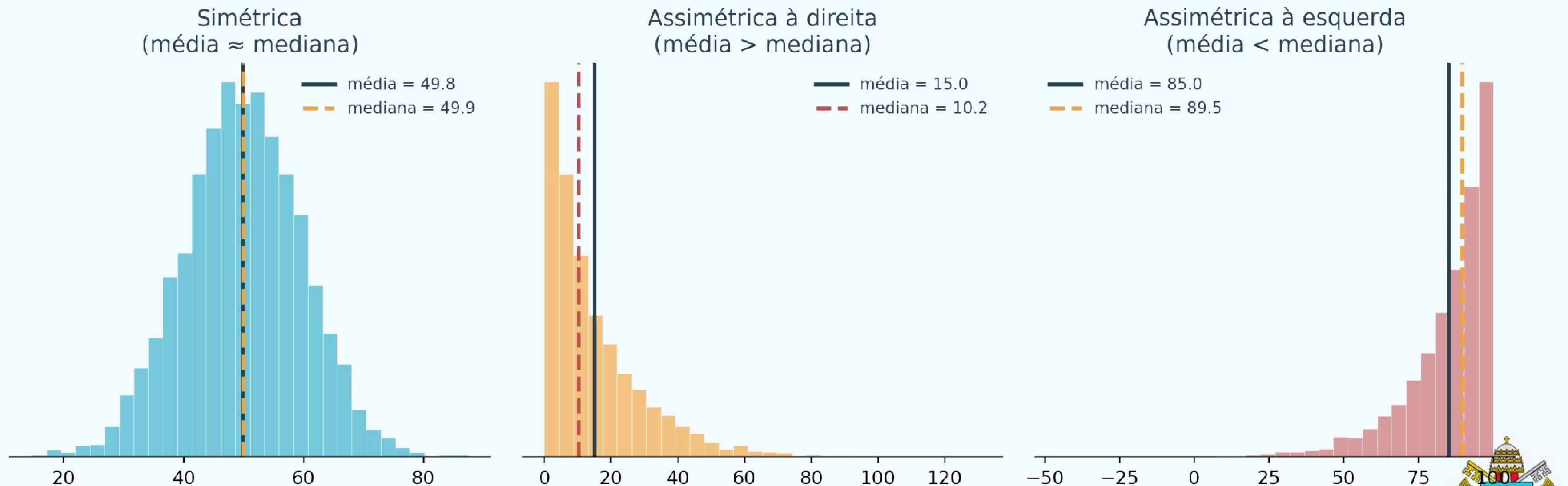


Histograma + KDE de tenure (dados ilustrativos)

Exemplo: "tenure" tem leve assimetria à direita: muitos clientes novos, poucos antigos. Algoritmos lineares sofrem com isso.

Simétrica, Assimétrica à Direita e à Esquerda

A posição relativa de média e mediana é o jeito mais rápido de diagnosticar a assimetria, sem nem olhar o gráfico.



Média = mediana na simétrica; em cauda longa à direita, a média é "puxada" para cima; em cauda longa à esquerda, para baixo.

Gerando o Histograma com Média e Mediana em Python

```
import seaborn as sns
import matplotlib.pyplot as plt

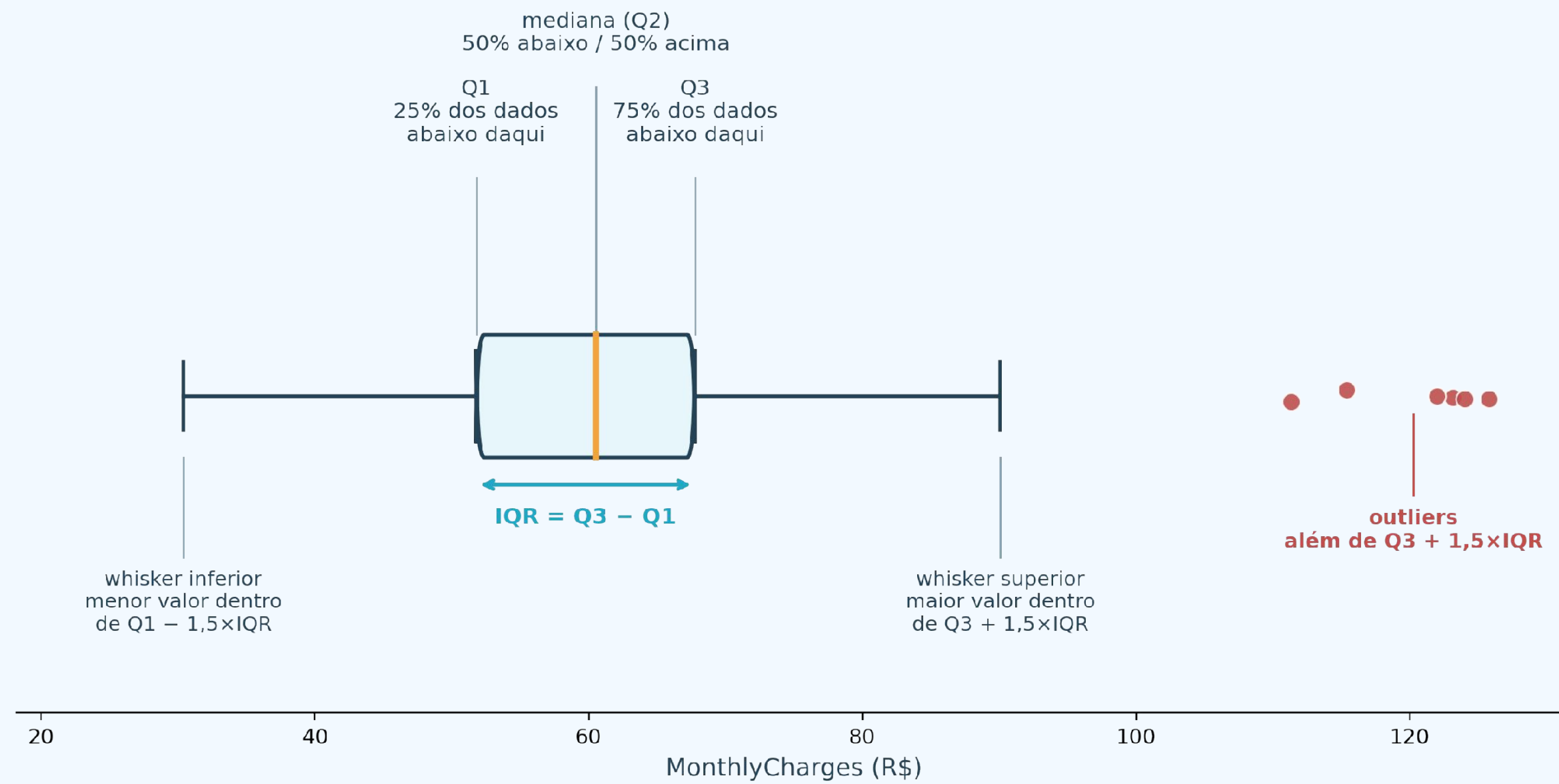
media = df['tenure'].mean()
mediana = df['tenure'].median()

sns.histplot(df['tenure'], kde=True, color='#23A8C3')
plt.axvline(media, color='#244357', label=f'média = {media:.1f}')
plt.axvline(mediana, color='#F2A53D', linestyle='--', label=f'mediana = {mediana:.1f}')
plt.legend()
plt.show()
```

É exatamente este código que você vai rodar na Prática 01 para cada variável numérica do dataset.

O Que é um Boxplot, Afinal?

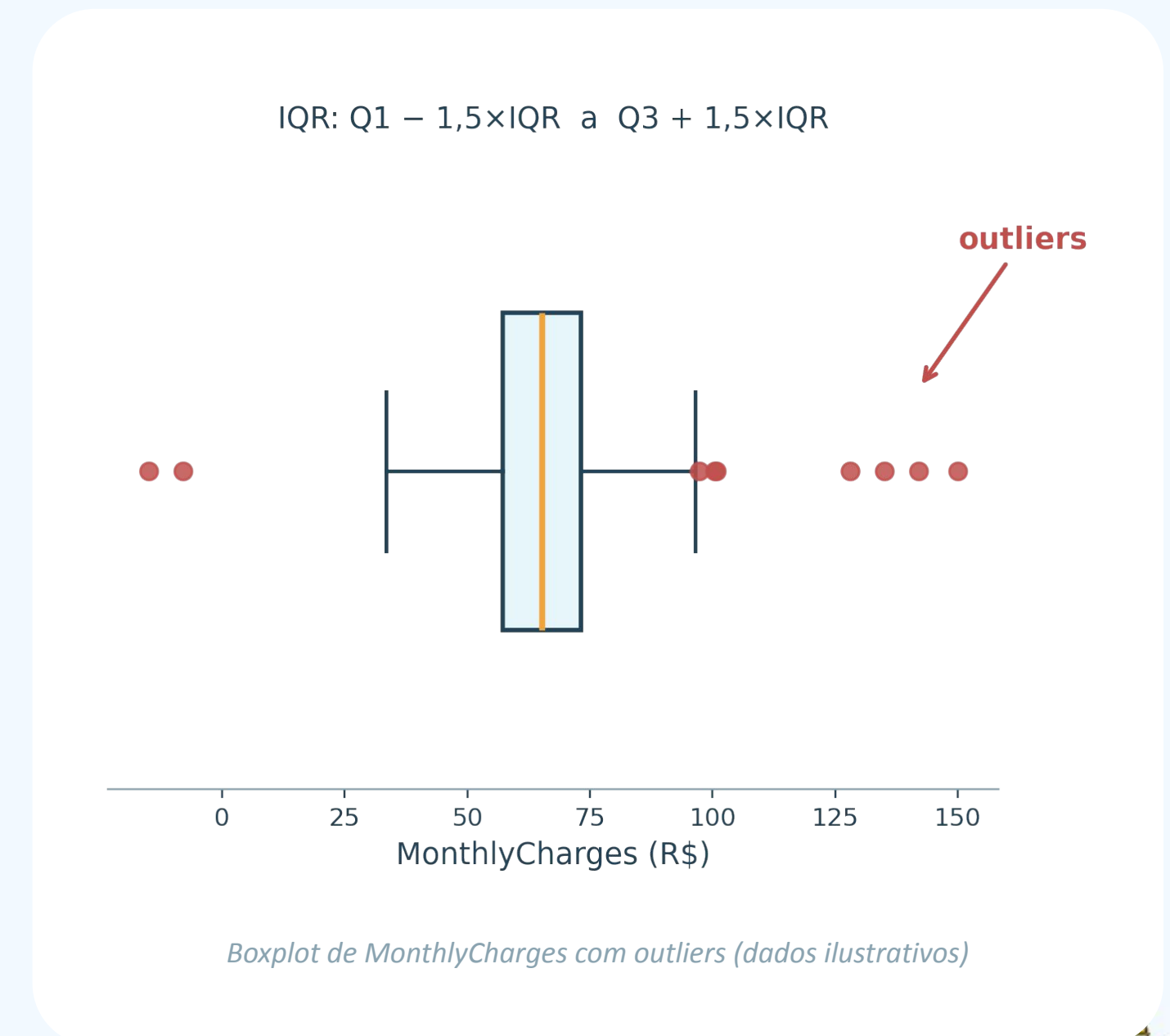
Antes de olhar outliers, vamos entender cada fronteira do boxplot: quartis, mediana, whiskers e IQR.



Q1, mediana e Q3 dividem os dados em quartis; os whiskers marcam até onde um valor ainda é "normal".

O Vilão ou o Protagonista?

- $IQR = Q3 - Q1$ · Limites: $[Q1 - 1,5 \times IQR, Q3 + 1,5 \times IQR]$
- Em preços de imóveis → outlier costuma distorcer o modelo.
- Em detecção de fraudes → o outlier É O SINAL que procuramos.



⚠ No nosso dataset: um cliente com tenure baixo e gasto alto pode ser insustentável: prestes a cancelar. Contexto é tudo.

Detectando Outliers com IQR em Python

```
Q1 = df['MonthlyCharges'].quantile(0.25)
Q3 = df['MonthlyCharges'].quantile(0.75)
IQR = Q3 - Q1

limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

outliers = df[(df['MonthlyCharges'] < limite_inferior) |
               (df['MonthlyCharges'] > limite_superior)]
print(f'{len(outliers)} outliers encontrados')
```

Mesmo código do boxplot acima, só que em números: os dois sempre devem contar a mesma história.

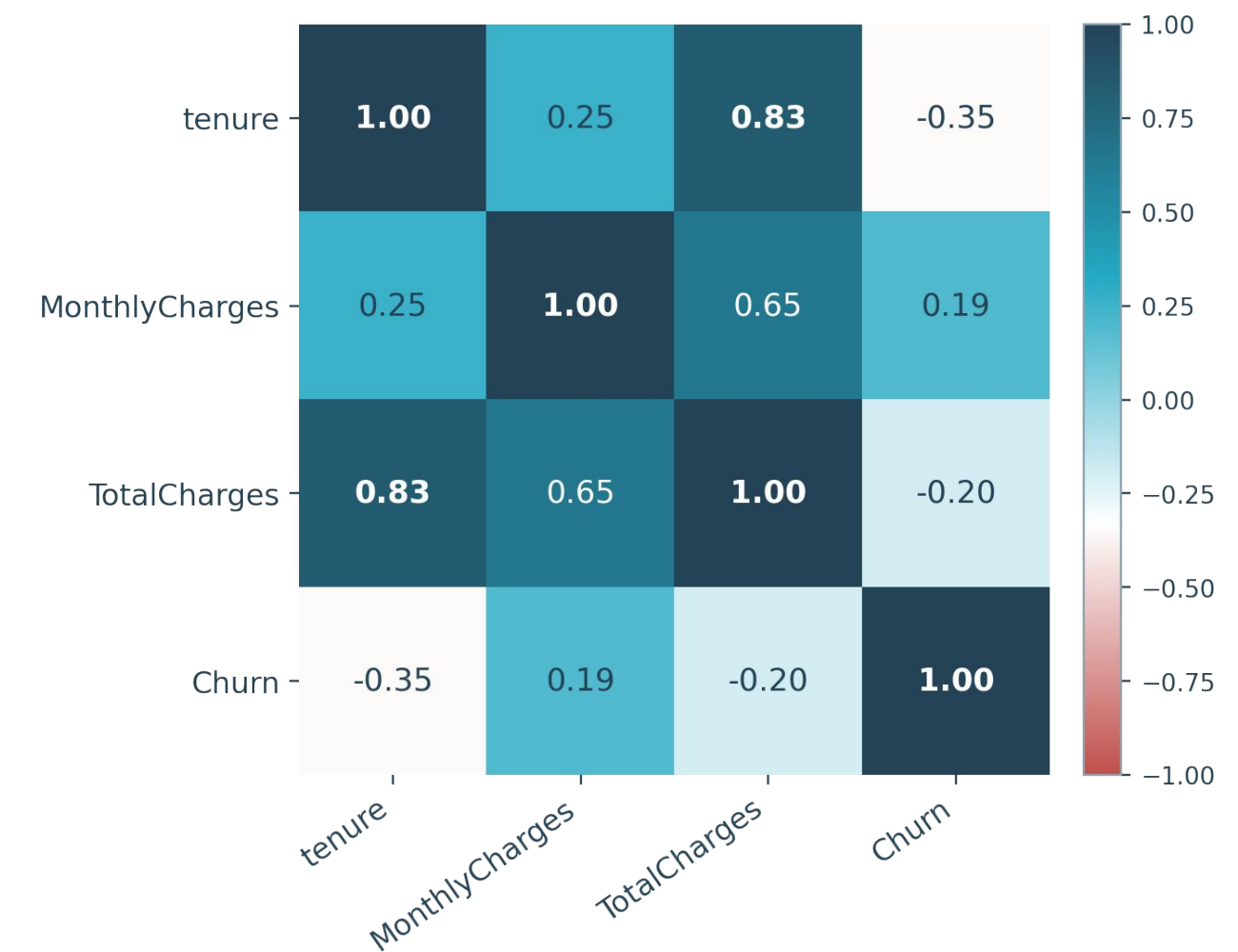
Quando Duas Variáveis Andam Juntas

Faixa de r (Pearson)	Interpretação
0,00 a $\pm 0,19$	Muito fraca ou nula
$\pm 0,20$ a $\pm 0,39$	Fraca
$\pm 0,40$ a $\pm 0,59$	Moderada
$\pm 0,60$ a $\pm 0,79$	Forte
$\pm 0,80$ a $\pm 1,00$	Muito forte

Pearson mede relação linear (numéricas contínuas); Spearman mede relação monotônica (ordinais ou não-normais).

Heatmap da Matriz de Correlação

- O heatmap é o "resumo executivo" de todas as correlações do dataset de uma vez.
- No nosso caso: tenure e TotalCharges têm correlação muito alta ($\approx 0,83$).
- Correlação alta entre features pode confundir modelos lineares: considere remover uma ou combiná-las.



Matriz de correlação (dados ilustrativos)

⚠ Correlação não é causalidade, mas é um ótimo ponto de partida para Feature Engineering.

Gerando o Heatmap de Correlação em Python

```
import seaborn as sns

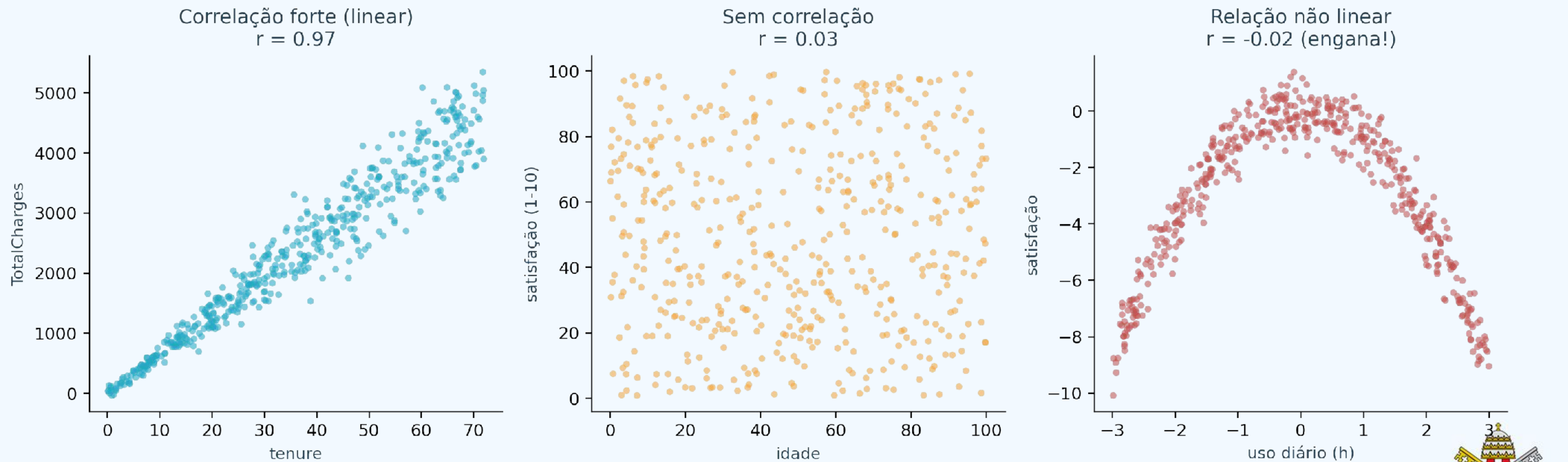
numericas = df.select_dtypes(include='number')
corr = numericas.corr()

sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            vmin=-1, vmax=1)
```

df.corr() calcula Pearson por padrão: para relações não lineares, use method='spearman'.

Scatter Plot: Vendo a Correlação (e Suas Armadilhas)

O heatmap resume tudo em um número: o scatter plot mostra a forma real da relação, e o número pode enganar.



À esquerda, r alto e relação linear de verdade. No meio, ruído puro. À direita, $r \approx 0$ mesmo havendo um padrão claríssimo, só que não linear.

Gerando o Scatter Plot em Python

```
import seaborn as sns

sns.scatterplot(data=df, x='tenure', y='TotalCharges',
               hue='Churn', alpha=0.6, palette=['#23A8C3', '#C0504D'])

# Dica: sns.pairplot(df[numerics + ['Churn']], hue='Churn')
# gera todos os pares de scatter plots de uma vez
```

Sempre cruze o scatter com o heatmap: um r baixo não significa ausência de relação, só de relação LINEAR.

Divisão Treino / Teste, Visualmente

DATASET COMPLETO — 7.043 clientes (100%)

▼ `train_test_split(test_size=0.3, stratify=y)`

TREINO — 70%

4.930 clientes

Churn: 26,5% Sim / 73,5% Não

usado para AJUSTAR o modelo
(`model.fit`)

TESTE — 30%

2.113 clientes

Churn: 26,5% Sim / 73,5% Não

usado só para AVALIAR
(`model.predict`)

stratify=y garante a MESMA proporção de churn nos dois conjuntos

O conjunto de teste fica "trancado": o modelo nunca o vê durante o treino, só na hora de avaliar.

Treino-Teste Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# stratify mantém a mesma proporção de churn em treino e teste
```

Testar o modelo com os mesmos dados do treino é como dar a prova com as respostas ao aluno.

Um outlier em um dataset de detecção de fraude deve ser sempre removido?

- A** Sim, outliers sempre distorcem o modelo e devem ser removidos.
- B** Não, em detecção de fraude, o outlier costuma ser exatamente o sinal que buscamos.
- C** Só se ele aparecer mais de uma vez no dataset.
- D** Depende apenas do tamanho do dataset.

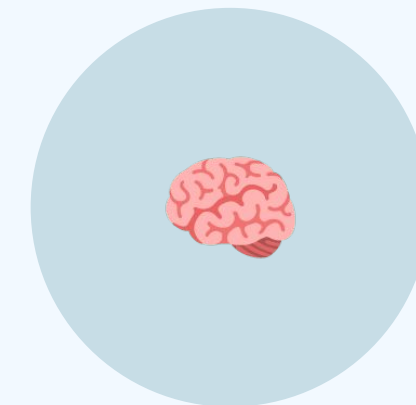
Um outlier em um dataset de detecção de fraude deve ser sempre removido?

- ☐ A Sim, outliers sempre distorcem o modelo e devem ser removidos.
- ☒ B Não, em detecção de fraude, o outlier costuma ser exatamente o sinal que buscamos.
- ☐ C Só se ele aparecer mais de uma vez no dataset.
- ☐ D Depende apenas do tamanho do dataset.

POR QUÊ? A decisão de tratar outliers é sempre uma decisão de negócio. Em fraude, o comportamento anômalo é o próprio alvo do modelo.

Feature Engineering com IA Generativa

- Criar novas variáveis a partir das existentes, usando conhecimento de domínio.
- **Estratégias clássicas:** datas → dia da semana; binning → idade → faixa etária; interações → ticket médio.
- O problema: sua criatividade sozinha é o limite. Um LLM pode sugerir 20 features em 30 segundos.



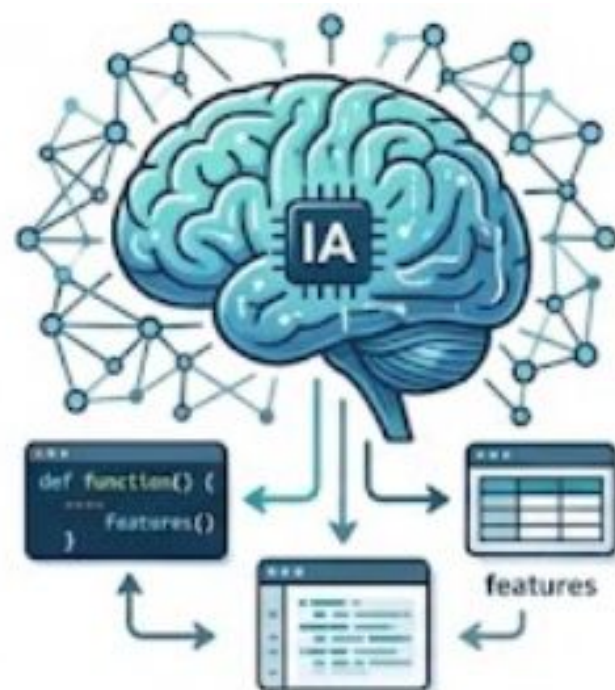
 A GenAI não substitui você: ela expande seu repertório de hipóteses. Você continua sendo o piloto.

PBL Aumentado: o Aluno como Auditor Técnico

1

IA sugere

A IA propõe features e código a partir do contexto do dataset.



2

Aluno valida

Faz sentido de negócio? É viável? Há risco de data leakage?



3

Aluno implementa

Só as melhores sugestões, auditadas, entram no pipeline.



Brainstorming de Features com IA

PROMPT

Atue como um Cientista de Dados Sênior
especialista em TELECOM.

Dataset: tenure, MonthlyCharges, TotalCharges,
Contract, PaperlessBilling, PaymentMethod.

Objetivo: prever CHURN.

Crie 5 NOVAS FEATURES que ajudem a prever churn.

Para cada uma, explique: nome e fórmula, por que
ajuda, código Pandas, e sinalize risco de DATA LEAKAGE.

SAÍDA DA IA (validada pelo auditor)

- Feature 1: $\text{avg_monthly_spend} = \text{TotalCharges} / \text{tenure}$
- Feature 2: $\text{is_high_risk_segment} = (\text{MonthlyCharges} > 70) \ \& \ (\text{Contract} == \text{'Mensal'})$
- Feature 3: $\text{tenure_to_charge_ratio}$

 **RISCO:** se $\text{tenure_to_charge_ratio}$ usar média GLOBAL, gera data leakage. Calcule apenas no treino.

Implementando as Features Sugeridas em Python

```
df_feat = df.copy()

# Feature 1: gasto médio mensal real (evita divisão por zero em tenure=0)
df_feat['avg_monthly_spend'] = df_feat['TotalCharges'] / df_feat['tenure'].replace(0, 1)

# Feature 2: segmento de alto risco (regra de negócio)
df_feat['is_high_risk_segment'] = (
    (df_feat['MonthlyCharges'] > 70) & (df_feat['Contract'] == 'Mensual')
)

# Feature 3: proporção entre tempo de casa e valor pago
df_feat['tenure_to_charge_ratio'] = df_feat['tenure'] / df_feat['MonthlyCharges']
```

Cada feature nasce de uma sugestão da IA, mas quem escreve e valida o código é você.

Checklist do Auditor de Features

Use este checklist para CADA feature sugerida pela IA: nunca confie cegamente.

- ☐ 1. Os dados para calcular a feature EXISTEM? (Se não → alucinação, descarte.)
- ☐ 2. A feature tem SENTIDO DE NEGÓCIO? (Um gestor concordaria?)
- ☐ 3. Não há DATA LEAKAGE? (Não usa informação do futuro ou do conjunto de teste.)
- ☐ 4. É COMPUTACIONALMENTE VIÁVEL? (Não vai explodir memória em produção?)

Estratégias para Missing Data

Remoção

- Linhas com poucos ausentes.
- Colunas com >70% de ausência.

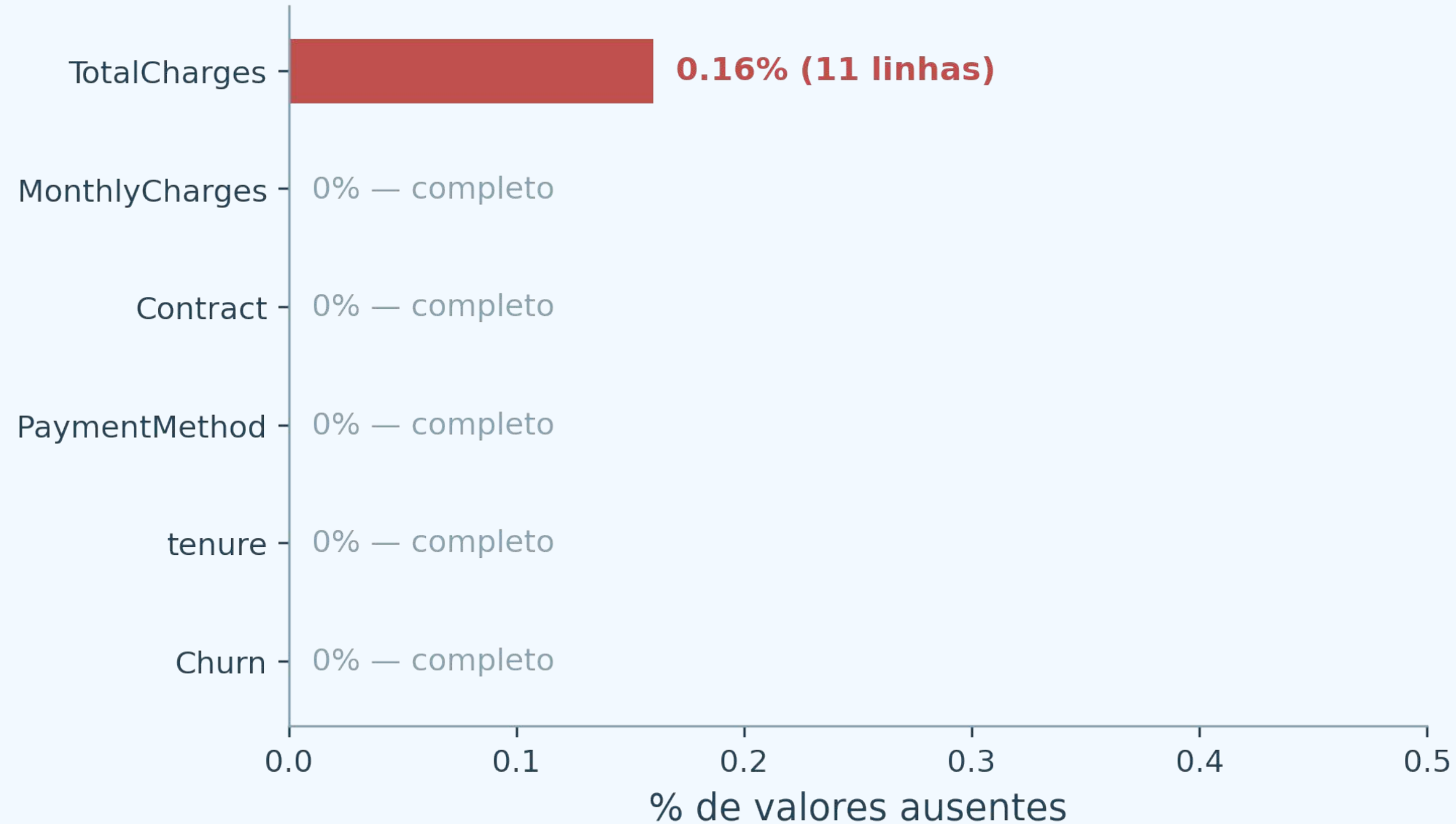
Imputação Simples

- Média, mediana ou moda.
- SimpleImputer do scikit-learn.

Imputação Avançada

- KNNImputer: usa vizinhos mais próximos para prever o valor.

Onde Estão os Dados Ausentes?



% de valores ausentes por coluna, dataset Telecom Churn (dados ilustrativos)

Ausência Também é Informação

- No nosso dataset, TotalCharges tem valores ausentes para clientes com tenure zero.
- Isso não é erro, é informação: são clientes novos. Podemos preencher com 0.
- Pergunte sempre: por que está ausente? É aleatório ou sistemático?



Imputando Dados Ausentes em Python

```
from sklearn.impute import SimpleImputer, KNNImputer

# Imputação simples: preenche com um valor fixo (0, média ou mediana)
imputer_simples = SimpleImputer(strategy='constant', fill_value=0)
df['TotalCharges'] = imputer_simples.fit_transform(df[['TotalCharges']])

# Alternativa: strategy='mean' ou strategy='median' para colunas numéricas

# Imputação avançada: usa os vizinhos mais parecidos para estimar o valor
imputer_knn = KNNImputer(n_neighbors=5)
df[['TotalCharges']] = imputer_knn.fit_transform(df[['TotalCharges']])
```

Assim como o scaler, o imputer também deve ser ajustado (fit) só no treino, para não vazar informação do teste.

Por Que Padronizamos os Dados?

Antes das fórmulas, a pergunta que importa: qual problema o escalonamento resolve?

- **Muitos algoritmos medem distância** (KNN, K-means, SVM) ou usam gradiente (regressão logística, redes neurais).
- Sem escalonamento, uma feature com valores até 5.000 (TotalCharges) domina outra até 5 (nº de dependentes), não porque é mais importante, só porque tem números maiores.
- Algoritmos baseados em árvore (Decision Tree, Random Forest, XGBoost) não precisam de escalonamento: eles comparam valores dentro da mesma coluna, não entre colunas.

⚠ Regra prática: se o algoritmo calcula distância ou gradiente, escalone. Se ele só faz splits (árvores), não precisa.

Padronização vs. Normalização

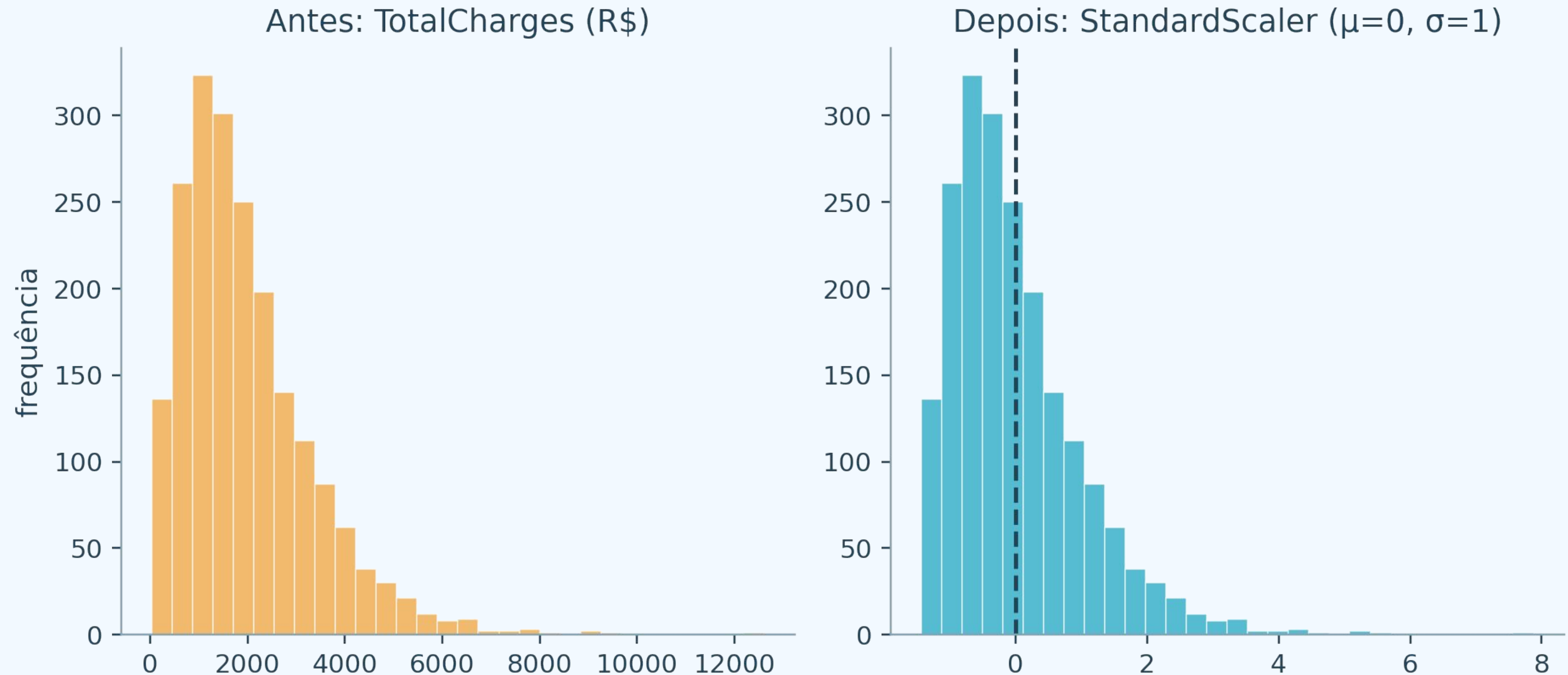
StandardScaler (Padronização)

- Transforma para média 0 e desvio padrão 1.
- $z = (x - \mu) / \sigma$
- Ideal para regressão, SVM, PCA.
- Mais robusto a outliers.

MinMaxScaler (Normalização)

- Comprime para o intervalo [0, 1].
- $x' = (x - \min) / (\max - \min)$
- Ideal para redes neurais e k-NN.
- Sensível a outliers.

Como o Escalonamento Transforma a Distribuição



TotalCharges antes e depois do StandardScaler (dados ilustrativos)

Escalonando as Variáveis em Python

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# fit_transform SÓ no treino
X_train[numericas] = scaler.fit_transform(X_train[numericas])

# no teste, apenas transform (usa média/desvio já aprendidos)
X_test[numericas] = scaler.transform(X_test[numericas])
```

Repare: fit_transform no treino, transform no teste, nunca o contrário, e nunca fit no dataset inteiro.

Encoding e Data Leakage

Técnica	Quando usar	Cuidado
One-Hot Encoding	Categorias nominais (sem ordem)	Aumenta dimensionalidade
Ordinal Encoding	Categorias com ranking natural	Preserva a ordem
Target Encoding	Alta cardinalidade	⚠ Alto risco de data leakage se feito antes do split
Binary Encoding	Cardinalidade muito alta (CEPs, IDs)	Reduz colunas com eficiência

🏆 REGRA DE OURO: `fit()` apenas no TREINO; `transform()` no treino E no teste. Nunca use `fit()` no dataset inteiro!

Codificando Variáveis Categóricas em Python

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# fit SÓ no treino
encoder.fit(X_train[categoricas])

train_encoded = encoder.transform(X_train[categoricas])
test_encoded = encoder.transform(X_test[categoricas])

# atalho equivalente para prototipagem rápida (cuidado com o split!):
# pd.get_dummies(df[categoricas], drop_first=True)
```

handle_unknown='ignore' evita erro se o teste tiver uma categoria que o treino nunca viu.

Por que Target Encoding é arriscado se aplicado antes do train/test split?

- A** Porque deixa o dataset muito grande.
- B** Porque o modelo passa a "ver" informação do conjunto de teste durante o treino (data leakage).
- C** Porque só funciona com variáveis numéricas.
- D** Porque exige GPU para ser calculado.

Por que Target Encoding é arriscado se aplicado antes do train/test split?

- ☐ A Porque deixa o dataset muito grande.
- ☒ B Porque o modelo passa a "ver" informação do conjunto de teste durante o treino (data leakage).
- ☐ C Porque só funciona com variáveis numéricas.
- ☐ D Porque exige GPU para ser calculado.

POR QUÊ? Target Encoding substitui a categoria pela média do alvo. Se calculado com o dataset inteiro, o treino "vê" estatísticas do teste: otimismo artificial que não se sustenta em produção.

EDA e Feature Engineering com GenAI

 Abra o notebook: [tarefa_1_eda.ipynb](#)

- Carregue o dataset de Telecom Churn e explore shape, tipos e valores ausentes.
- Gere histogramas, boxplots e um heatmap de correlação.
- Aplique o prompt de brainstorming de features a um LLM de sua escolha.
- Use o Checklist do Auditor e implemente as 3 melhores features sugeridas.
- Compare um modelo simples com e sem as novas features.

 45 minutos · Individual · Documente os prompts usados (prompt log)

Resumo da Unidade 1

Tópico	O que levar para o projeto
Ciclo de vida	Todo projeto de ML começa no problema de negócio, não no algoritmo.
EDA	Conhecer os dados antes de modelar evita erros caros depois.
Feature Eng. com GenAI	LLMs aceleram o brainstorming; o aluno audita cada sugestão.
Pré-processamento	Limpeza, escalonamento e encoding sem data leakage.
Próximo passo	Unidade 2: usaremos essas features para construir modelos de regressão e ensembles.

Referências Bibliográficas

Referência
GÉRON, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. 3.ed. O'Reilly, 2022.
MOLNAR, C. Interpretable Machine Learning. 2.ed. 2022.
HUYEN, C. Designing Machine Learning Systems. O'Reilly, 2022.
MÜLLER, A.; GUIDO, S. Introduction to Machine Learning with Python. O'Reilly, 2016.
RASCHKA, S.; MIRJALILI, V. Machine Learning with PyTorch and Scikit-Learn. Packt, 2022.
WHITE, J. et al. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. arXiv, 2023.